Clase 5 · Estados de un proceso, señales, wait() y zombies

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · martes 8 sep 2026 · Tema 2 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

Fuente

No asistí (entrevista). Reconstruida de la Práctica #3 "Procesos (continuación)" del sitio del profesor. Confirmado por el repaso de la Clase 6: aquí arrancó el Tema 2, se dejó revisar `man 7 signal` (columna x86/ARM), se demostró la adopción de huérfanos (PPID → 1) y **SIGCHLD es 17** en x86/ARM.

1. Estados de un proceso

Modo usuario o modo núcleo; los estados existen porque los procesos compiten por la CPU.

1. Ejecución en modo usuario.
2. Ejecución en modo núcleo.
3. Listo y en memoria.
4. Dormido y en memoria (espera un suceso).
5. Listo y descargado (hay que cargarlo a memoria principal).
6. Dormido y descargado (en almacenamiento secundario).
7. Expulsado (vuelve a usuario pero hay cambio de contexto).
8. Creado (por `fork`, aún no listo).
9. **Zombie** (ya no existe, deja un registro para el padre).

Transiciones: 1 → 2 por llamada al sistema o interrupción; 2 → 7 → 1; 2 → 9 por `exit`; 8 → 3 o 5 según haya memoria; 3 ↔ 4 ↔ 6 ↔ 5 entre memoria principal y secundaria.

Contexto: código, variables globales, registros de la CPU, entrada en la tabla de procesos, área de usuario y pilas.

2. Señales

Mensajes **asíncronos** (se atienden de inmediato); se usan por nombre; `<signal.h>`. SIGBUS, SIGSEGV y SIGFPE terminan con core; SIGTERM y SIGKILL desde otro proceso; SIGHUP despierta. El manejador hace lo mínimo y regresa.

Señal	Núm. (System V)	Nota
SIGHUP	1	Terminal desconectada
SIGINT	2	Ctrl+C
SIGQUIT	3	Ctrl+\, core

SIGILL	4	Instrucción ilegal
SIGTRAP	5	Paso a paso
SIGIOT	6	`abort()`
SIGEMT	7	Emulator trap
SIGFPE	8	Coma flotante
SIGKILL	**9**	No se puede ignorar
SIGBUS	10	Dirección inexistente o desalineada
SIGSEGV	11	Violación de segmento
SIGSYS	12	Argumento erróneo
SIGPIPE	13	Tubería sin lector
SIGALRM	14	Temporizador en cero
SIGTERM	**15**	Se puede manejar (limpiar antes de salir); se manda en el shutdown
SIGUSR1 / 2	16 / 17	Para el programador
SIGCHLD	18 (System V)	**17 en x86/ARM**; hijo → padre; ignorada por defecto
SIGPWR	19	Falla de alimentación

3. wait()

Bloquea al padre hasta que el hijo termina y devuelve un estado:

Macro	Significado
WIFEXITED / WEXITSTATUS	Terminó con exit / los 8 bits bajos de su código
WIFSIGNALED / WTERMSIG	Terminó por señal / cuál
WCOREDUMP	Generó core
WIFSTOPPED / WSTOPSIG	Está parado / por qué señal

`fork-exec.c`: `spawn("ls", {"ls","-l","/"})` hace `fork` + `execvp`; el padre llama `wait(&child_status)` e imprime el código si `WIFEXITED`.

4. Zombies

Proceso terminado que sigue en el sistema porque el padre no llamó `wait()`. `zombie.c`: el hijo hace `exit(0)` y el padre `sleep(60)`; con `ps -e -o pid,ppid,stat,cmd` se ve **Z+ <defunct>**.

5. kill() y abort()

- Terminación normal o por señal: SIGINT, SIGTERM (`kill`), SIGABRT (`abort()`, core), SIGKILL (no se bloquea ni se maneja).
- Terminal: `kill -SEÑAL PID`. Código: `kill(child_pid, SIGTERM)`.

- `kill.c`: el hijo imprime su PID cada segundo; tras `sleep(10)` el padre lo termina con SIGTERM.
- `abort.c`: el proceso se manda SIGABRT e imprime "Aborted".
- Código de salida (`$?`): 0 = bien; 0–127 normal; **≥ 128 = interrumpido por señal**.

Pendiente (cuestionario de la Práctica 3)

Con `kill.c`, crear otro hijo y explicar qué pasa; hacer que termine a los 20 s.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Concepto	Para recordar
Estados	9; el 9 es zombie
Señales	Asíncronas; KILL 9 ineludible, TERM 15 manejable
SIGCHLD	17 en x86/ARM (18 en la tabla System V)
wait()	Bloquea al padre; WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG
Zombie	Padre no llamó wait(); `ps`: Z+ defunct
`$?`	≥ 128 = terminó por señal